

```
askFeeling()
print('Good afternoon!')
askFeeling()
print('Good evening!')
askFeeling()
```

In this next example, I've moved the duplicate code into a loop:

```
for timeOfDay in ['morning', 'afternoon', 'evening']:
    print('Good ' + timeOfDay + '!')
    print('How are you feeling?')
    feeling = input()
    print('I am happy to hear that you are feeling ' + feeling + '.')
```

You could also combine these two techniques and use a function and a loop:

```
def askFeeling(timeOfDay):
    print('Good ' + timeOfDay + '!')
    print('How are you feeling?')
    feeling = input()
    print('I am happy to hear that you are feeling ' + feeling + '.')

for timeOfDay in ['morning', 'afternoon', 'evening']:
    askFeeling(timeOfDay)
```

Notice that the code that produces the “Good morning/afternoon/evening!” messages is similar but not identical. In the third improvement to the program, I parameterized the code to deduplicate the identical parts. Meanwhile, the `timeOfDay` parameter and `timeOfDay` loop variable replace the parts that differ. Now that I've deduplicated this code by removing the extra copies, I only need to make any necessary changes in one place.

As with all code smells, avoiding duplicate code isn't a hard-and-fast rule you must always follow. In general, the longer the duplicate code section or the more duplicate copies that appear in your program, the stronger the case for deduplicating it. I don't mind copying and pasting code once or even twice. But I generally start to consider deduplicating code when three or four copies exist in my program.